

# Needleman-Wunsch Algorithm

Accelerating Global DNA Sequence Alignment  
using OpenMP Anti-Diagonal Parallelisation



**Presented By**

Mian Saad Tahir

2023-CS-62

**Supervised by**

Sir Waqas Ali

**Course**

Parallel and Distributed Computing

**Department of Computer Science**

**University of Engineering and Technology, Lahore**

**April 2026**

May 17, 2026

# Contents

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>Problem Statement</b>               | <b>3</b> |
| <b>2</b> | <b>Background</b>                      | <b>3</b> |
| 2.1      | Scoring Model . . . . .                | 3        |
| 2.2      | Recurrence Relation . . . . .          | 3        |
| 2.3      | Worked Example . . . . .               | 4        |
| <b>3</b> | <b>Sequential Implementation</b>       | <b>4</b> |
| <b>4</b> | <b>Parallelisation Strategy</b>        | <b>5</b> |
| 4.1      | The Anti-Diagonal Wavefront . . . . .  | 5        |
| 4.2      | Parallel Environment: OpenMP . . . . . | 5        |
| <b>5</b> | <b>Expected Outcomes</b>               | <b>5</b> |
| 5.1      | Performance . . . . .                  | 5        |
| 5.2      | Speedup . . . . .                      | 6        |

# 1 Problem Statement

DNA sequence alignment is one of the most computationally demanding tasks in modern bioinformatics. Given two biological sequences, the goal is to find the optimal global alignment, identifying matches, mismatches, and gaps to quantify their similarity. This underpins critical real-world applications including disease mutation detection, COVID-19 variant tracking, cancer genomics, and ancestry analysis tools such as 23andMe.

The **Needleman-Wunsch algorithm** [1] solves the global sequence alignment problem optimally using dynamic programming. It builds an  $(M + 1) \times (N + 1)$  scoring matrix where  $M$  and  $N$  are the lengths of the two input sequences. Each cell is computed from three neighbours, making the sequential version inherently  $O(MN)$  in both time and space.

## The Bottleneck

For real genomic sequences, lengths commonly reach  $N = 30,000$  or more, producing scoring matrices with **900 million cells**. The sequential implementation processes these one cell at a time, taking tens of minutes per alignment. In a clinical setting where thousands of sequences must be compared daily, this is a critical performance bottleneck.

# 2 Background

## 2.1 Scoring Model

The algorithm assigns a numerical score to each alignment decision:

Table 1: Needleman-Wunsch Scoring Parameters

| Event    | Description                      | Score |
|----------|----------------------------------|-------|
| Match    | Same character at both positions | +1    |
| Mismatch | Different characters             | -1    |
| Gap      | Inserting a dash in one sequence | -2    |

## 2.2 Recurrence Relation

Each cell  $H[i][j]$  in the scoring matrix is computed as:

$$H[i][j] = \max \begin{cases} H[i-1][j-1] + s(a_i, b_j) & \text{(diagonal: match or mismatch)} \\ H[i-1][j] + g & \text{(up: gap in sequence 2)} \\ H[i][j-1] + g & \text{(left: gap in sequence 1)} \end{cases} \quad (1)$$

where  $s(a_i, b_j)$  is the match/mismatch score and  $g = -2$  is the gap penalty. Boundary conditions are  $H[i][0] = i \cdot g$  and  $H[0][j] = j \cdot g$ .

## 2.3 Worked Example

|   | –  | A  | T  | C  | G  |
|---|----|----|----|----|----|
| – | 0  | -2 | -4 | -6 | -8 |
| A | -2 | 1  | -1 | -3 | -5 |
| T | -4 | -1 | 2  | 0  | -2 |
| C | -6 | -3 | 0  | 3  | 1  |

Figure 1: Scoring matrix for ATC vs ATCG. Final score = 1, traceback gives alignment ATC– / ATCG.

## 3 Sequential Implementation

The sequential baseline fills the scoring matrix using two nested loops. A direction matrix of the same size records whether each cell was reached via diagonal, up, or left — used during traceback to reconstruct the alignment. Complexity is  $O(MN)$  time and  $O(MN)$  space.

```

1 // Scoring constants
2 const int MATCH      = 1;
3 const int MISMATCH   = -1;
4 const int GAP        = -2;
5
6 // Matrix stored as flat 1D vector (cache-friendly row-major)
7 // Access cell (i,j) as: matrix[i * (N+1) + j]
8 int needleman_wunsch_seq(const string& A, const string& B,
9                           vector<int>& matrix,
10                          vector<int>& direction) {
11     int M = A.size(), N = B.size();
12     matrix.assign((M+1)*(N+1), 0);
13     direction.assign((M+1)*(N+1), 0);
14
15     // Initialise first row and column with gap penalties
16     for (int i = 0; i <= M; i++) matrix[i*(N+1)] = i * GAP;
17     for (int j = 0; j <= N; j++) matrix[j] = j * GAP;
18
19     // Fill matrix row by row      purely sequential
20     for (int i = 1; i <= M; i++) {
21         for (int j = 1; j <= N; j++) {
22             // Score for matching/mismatching A[i-1] with B[j-1]
23             int match = (A[i-1] == B[j-1]) ? MATCH : MISMATCH;
24
25             int diag = matrix[(i-1)*(N+1)+(j-1)] + match;
26             int up   = matrix[(i-1)*(N+1)+j] + GAP;
27             int left = matrix[i*(N+1)+(j-1)] + GAP;
28
29             // Take the best of three choices
30             if (diag >= up && diag >= left) {
31                 matrix[i*(N+1)+j] = diag;
32                 direction[i*(N+1)+j] = 0; // diagonal
33             } else if (up >= left) {
34                 matrix[i*(N+1)+j] = up;
35                 direction[i*(N+1)+j] = 1; // up
36             } else {
37                 matrix[i*(N+1)+j] = left;

```

```

38         direction[i*(N+1)+j] = 2; // left
39     }
40 }
41 }
42
43 return matrix[M*(N+1)+N]; // final alignment score
44 }

```

Listing 1: Sequential Needleman-Wunsch core (C++)

## 4 Parallelisation Strategy

### 4.1 The Anti-Diagonal Wavefront

The fundamental dependency in the scoring matrix is that each cell  $H[i][j]$  requires  $H[i-1][j-1]$ ,  $H[i-1][j]$ , and  $H[i][j-1]$ . This prevents parallelising rows or columns directly. However, cells along the same **anti-diagonal** (where  $i + j = \text{constant}$ ) have *no dependencies on each other* — they only depend on the previous anti-diagonal, which is already complete.



Figure 2: Anti-diagonal indices ( $i + j$ ) for a  $4 \times 4$  matrix. Cells sharing the same colour belong to the same anti-diagonal and can be computed in parallel.

### 4.2 Parallel Environment: OpenMP

The implementation will use **OpenMP** (Open Multi-Processing) on a shared-memory multi-core CPU. OpenMP was chosen over MPI because:

- All cells share the same scoring matrix in memory — no data distribution or message passing is needed.
- The anti-diagonal pattern maps directly to a single `#pragma omp parallel for` directive per diagonal.
- OpenMP is available by default on Linux/macOS with GCC.

## 5 Expected Outcomes

### 5.1 Performance

Table 2 shows the expected speedup based on the anti-diagonal parallelism theory and typical OpenMP performance on an 8-core machine.

Table 2: Expected Performance on 8-Core Machine (OpenMP)

| N    | Matrix Cells | Sequential (s) | Parallel (s) | Expected Speedup |
|------|--------------|----------------|--------------|------------------|
| 500  | 250,000      | $\sim 0.04$    | $\sim 0.009$ | $\sim 4.5\times$ |
| 1000 | 1,000,000    | $\sim 0.16$    | $\sim 0.03$  | $\sim 5.0\times$ |
| 2000 | 4,000,000    | $\sim 0.65$    | $\sim 0.14$  | $\sim 4.7\times$ |
| 4000 | 16,000,000   | $\sim 2.60$    | $\sim 0.55$  | $\sim 4.7\times$ |

## 5.2 Speedup

Even with 8 threads, the expected speedup is approximately  $4\text{--}5\times$  rather than  $8\times$ . This is explained by:

- **Amdahl's Law:** The initialisation, traceback, and boundary conditions are serial and cannot be parallelised.
- **Short diagonals:** The first and last  $(N - 1)$  diagonals have fewer than  $N$  cells, limiting parallelism at the corners.
- **Thread overhead:** OpenMP thread creation and barrier synchronisation add a small fixed cost per diagonal.
- **Cache pressure:** Multiple threads writing to the same matrix row simultaneously can cause cache line contention.

Speedup is expected to improve with larger  $N$  as the parallel fraction of work grows relative to the fixed overheads.

## References

- [1] S.B. Needleman and C.D. Wunsch, *A general method applicable to the search for similarities in the amino acid sequence of two proteins*, Journal of Molecular Biology, 48(3):443–453, 1970.
- [2] P. Pacheco, *An Introduction to Parallel Programming*, Morgan Kaufmann, 2011.
- [3] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, Version 5.2, 2021. <https://www.openmp.org/specifications/>
- [4] S.F. Altschul et al., *Basic local alignment search tool*, Journal of Molecular Biology, 215(3):403–410, 1990.